

PROJECT

7.3

SECURE PACKAGES WITH CODEARTIFACT

DEVOPS SERIES

Prepared by:
Amrize JK
Certified Solutions Architect Associate

Objective

This project sets up a secure workflow to publish and manage software packages from a web app hosted on EC2 using AWS CodeArtifact. GitHub is integrated for version control and collaboration.

IAM policies are used to control access, and CodeArtifact serves as a private package repository. The setup enables a CI-friendly environment for securely pushing and retrieving packages.

Services

Amazon EC2 - Hosts the web app and handles package publishing.

AWS IAM - Manages secure access to AWS resources.

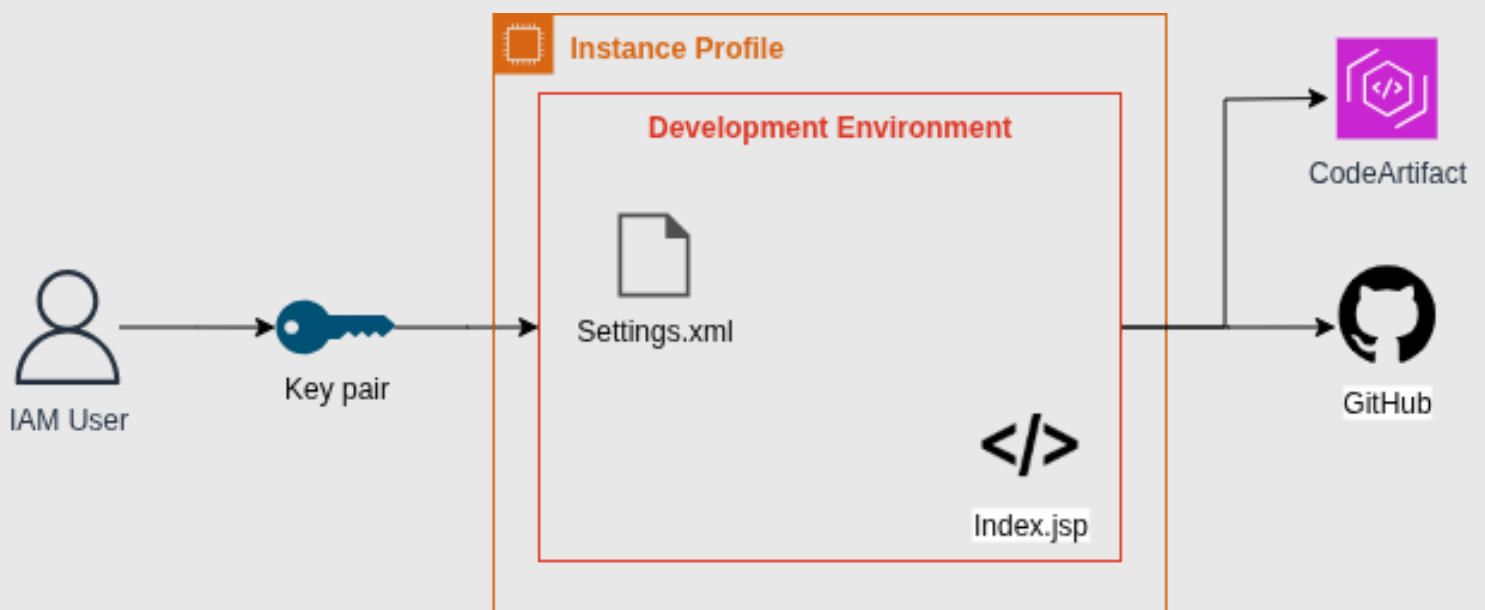
AWS CodeArtifact - Stores and retrieves private packages.

GitHub - Manages source code and collaboration.

AWS CLI - Connects and operates AWS services from the terminal.

Git - Tracks code and pushes updates to GitHub.

Architecture Diagram



This architecture shows a secure CI workflow where an IAM user accesses a development environment on an EC2 instance via an instance profile.

The application code (e.g., index.jsp) is managed locally, and a settings.xml file is used to configure access to AWS CodeArtifact for pulling dependencies. GitHub is connected for version control, and IAM ensures secure access throughout the setup.

Steps to Implement

1. Launch and prepare your EC2 instance:

Start an EC2 instance, connect via VSCode Remote-SSH, and set up your app environment for deployment.

2. Initialize Git and connect to GitHub:

Create a Git repository in your EC2 instance, link it to GitHub using `git remote add origin`.

3. Generate and use a GitHub PAT:

Create a Personal Access Token from GitHub, and use it as your password during the git push operation for secure authentication.

4. Install and configure the AWS CLI:

Download AWS CLI on EC2, configure it using your access keys, and test the connection with `aws s3 ls`.

5. Set up AWS CodeArtifact repository:

Create a domain and repository in CodeArtifact to store and manage your private packages securely.

6. Create IAM policy and attach to EC2:

Define a policy for CodeArtifact access, and attach it to the IAM role linked to your EC2 instance.

7. Authenticate and publish to CodeArtifact:

Use AWS CLI to log in to CodeArtifact, configure your package manager, and publish your package securely.

Story Time

EC2 (The Cloud Castle for Your App):

EC2 is your grand castle in the cloud where your web app lives. You summon it (launch it), decorate it (configure it), and then connect from afar like a wizard using VSCode's magic portal (Remote-SSH).

GitHub (The Code Scroll Library):

GitHub is your enchanted library that stores ancient scrolls of code. You link your EC2 castle to it, so every spell (code update) you write is recorded and shared with your fellow developers.

Git (The Spell Recorder):

Git is your loyal scribe who records every change you make in your app. It helps you travel back in time if things go wrong—and sends your spell updates straight to the GitHub library.

AWS CLI (The Magic Wand):

AWS CLI is your command-line wand that casts spells to control AWS services. With a few typed incantations, you can talk to CodeArtifact, manage secrets, or summon infrastructure.

CodeArtifact (The Package Vault):

CodeArtifact is your private treasure vault where you store magical packages (libraries). Only trusted heroes can add or retrieve potions (code packages) for their quests (projects).

IAM (The Gatekeeper of Permissions):

IAM is your mighty gatekeeper who hands out magical badges (permissions) to chosen warriors (users). Only those with the right badge can enter the EC2 realm and command cloud resources.

BY
AMRIZE J.K

THANK YOU!

